



Digital Design

Chapter 2: Combinational Logic Design

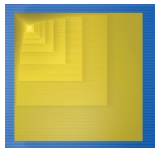
Slides to accompany the textbook *Digital Design*, First Edition,
by Frank Vahid, John Wiley and Sons Publishers, 2007.
<http://www.ddvahid.com>

Copyright © 2007 Frank Vahid

Instructors of courses requiring Vahid's *Digital Design* textbook (published by John Wiley and Sons) have permission to modify and use these slides for customary course-related activities, subject to keeping this copyright notice in place and unmodified. These slides may be posted as unanimated pdf versions on publicly-accessible course websites.. PowerPoint source (or pdf with animations) may not be posted to publicly-accessible websites, but may be posted for students on internal protected sites or distributed directly to students by other electronic means. Instructors may make printouts of the slides available to students for a reasonable photocopying charge, without incurring royalties. Any other use requires explicit permission. Instructors may obtain PowerPoint source or obtain special use permissions from Wiley – see <http://www.ddvahid.com> for information.

Boolean Algebra Terminology

- Example equation: $F(a,b,c) = a'bc + abc' + ab + c$
- **Variable**
 - Represents a value (0 or 1)
 - Three variables: a, b, and c
- **Literal**
 - Appearance of a variable, in true or complemented form
 - Nine literals: a' , b, c, a, b, c' , a, b, and c
- **Product term**
 - Product of literals (or a single literal)
 - Four product terms: $a'bc$, abc' , ab, c
- **Sum term**
 - Sum of literals (or a single literal), $a+b+c'$



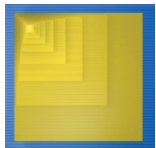
Boolean Algebra Terminology

- ***Sum-of-products***

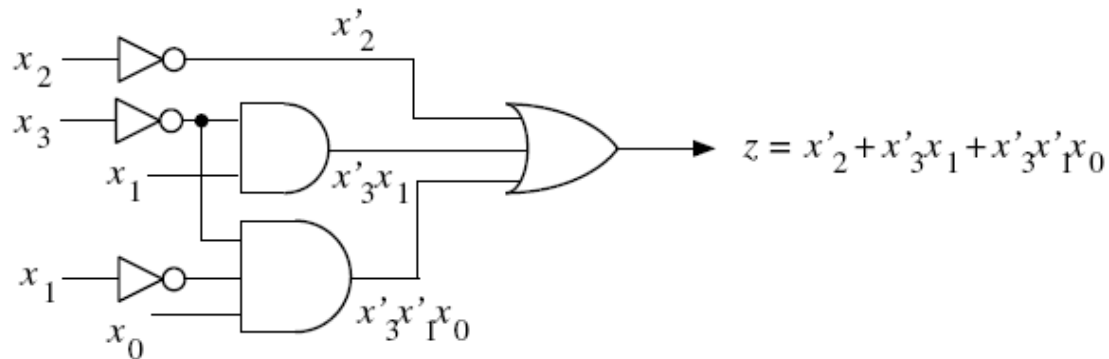
- Equation written as OR of product terms only
- Above equation is in sum-of-products form. “ $F = (a+b)c + d$ ” is not.
- $F = (a')' + b$ is not. $F = (ab + bc)'$ is not.

- ***Product-of-sums***

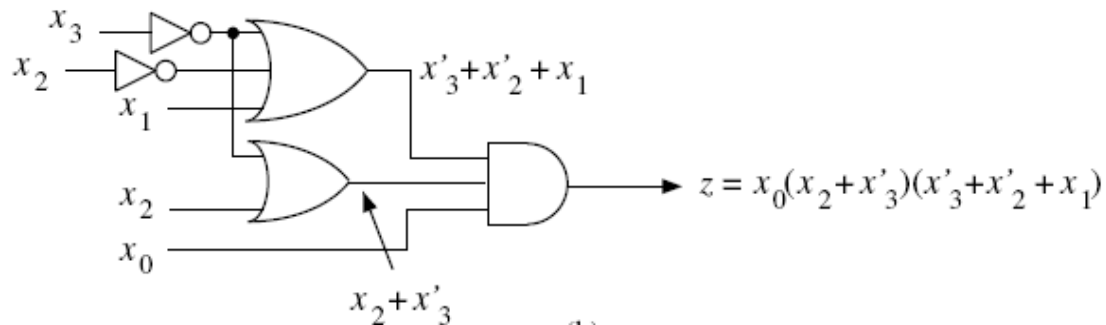
- Equation written as AND of sum terms only
- $F = (a+b+c)(a' + c')$
- $F = (a')' + b$ is not.



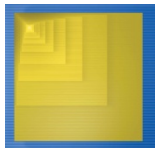
Sum of Products, Product of Sums



AND/OR



OR/AND



Standard Representation: Truth Table

- Many possible circuit and eqn representations of a function.
- How can we determine if two functions are the same?
 - Recall automatic door example
 - Same as $f = hc' + h'pc'$?
 - Used algebraic methods
 - But if we failed, does that prove *not* equal? No.
- Solution: Convert to truth tables
 - Only ONE truth table representation of a given function
 - **Standard** representation -- for given function, only one version in standard form exists

$$f = c'hp + c'hp' + c'h'p$$

$$f = c'h(p + p') + c'h'p$$

$$f = c'h(1) + c'h'p$$

$$f = c'h + c'h'p$$

(what if we stopped here?)

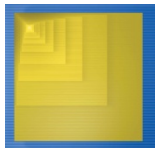
$$f = hc' + h'pc'$$

Q: Determine if $F=ab+a'$ is same function as $F=a'b'+a'b+ab$, by converting each to truth table first

F = ab + a'			F = a'b' + a'b + ab		
a	b	F	a	b	F
0	0	1	0	0	1
0	1	1	0	1	1
1	0	0	1	0	0
1	1	1	1	1	1

Same

a

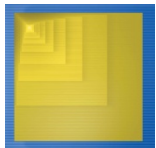


Truth Table Canonical Form

- Q: Determine via truth tables whether $ab + a'$ and $(a+b)'$ are equivalent

$F = ab + a'$			$F = (a+b)'$		
a	b	F	a	b	F
0	0	1	0	0	1
0	1	1	0	1	0
1	0	0	1	0	0
1	1	1	1	1	0

Not equivalent



Canonical Form -- Sum of Minterms

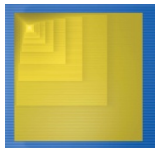
- Truth tables too big for numerous inputs
- Use standard form of equation instead
 - Known as **canonical form**
 - Regular algebra: group terms of polynomial by power
 - $ax^2 + bx + c$ ($3x^2 + 4x + 2x^2 + 3 + 1 \rightarrow 5x^2 + 4x + 4$)
 - Boolean algebra: create **sum of minterms**
 - **Minterm**: product term with every function literal appearing **exactly once**, in true or complemented form (aa' , aa not minterms)
 - Just multiply-out equation until sum of product terms
 - Then expand each term until all terms are minterms

Q: Determine if $F(a,b)=ab+a'$ is same function as $F(a,b)=a'b'+a'b+ab$, by converting first equation to canonical form (second already in canonical form)

$F = ab+a'$ (already sum of products)

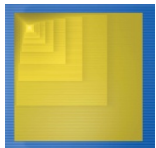
$F = ab + a'(b+b')$ (expanding term)

$F = ab + a'b + a'b'$ (SAME -- same three terms as other equation)



Exercise

- Convert $F(a,b,c) = (a+b)(a' + ac)b$ into sum-of-minterms.
- $F = (a+b)(a' + ac)b$ *Multiply out into sum of products*
 - = $(a+b)(a' b + acb)$ (distributivity)
 - = $a(a' b + acb) + b(a' b + acb)$ (distributivity)
 - = $aa' b + aacb + ba' b + bacb$ (distributivity)
 - = $0b + aacb + ba' b + bacb$ (complement)
 - = $0b + aacb + a' bb + acbb$ (commutativity)
 - = $0b + acb + a' b + acb$ (idempotent)
 - = $acb + a' b + acb$ (null element)
 - = $acb + a' b$ (idempotent) *Now in sum of products form*
 - = $acb + a' b1$ (identity) *expand each term*
 - = $acb + a' b(c+c')$ (complement)
 - = $acb + a' bc + a' bc'$ (distributivity)



Canonical Form – Sum of Minterms

- Q: Determine whether the functions $G(a,b,c,d,e) = abcd + a'bcde$ and $H(a,b,c,d,e) = abcde + abcde' + a'bcde + a'bcde(a' + c)$ are equivalent.

$$G = abcd + a'bcde$$

$$G = abcd(e+e') + a'bcde$$

$$G = abcde + abcde' + a'bcde$$

$$G = a'bcde + abcde' + abcde \quad (\text{sum of minterms form})$$

Equivalent

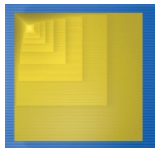
$$H = abcde + abcde' + a'bcde + a'bcde(a' + c)$$

$$H = abcde + abcde' + a'bcde + a'bcdea' + a'bcdec$$

$$H = abcde + abcde' + a'bcde + a'bcde + a'bcde$$

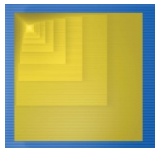
$$H = abcde + abcde' + a'bcde$$

$$H = a'bcde + abcde' + abcde$$



Canonical Form – Product of Maxterms

- Alternate Canonical Form
 - Boolean algebra: *product of maxterms*
 - **Maxterm**: sum term with every function variable appearing **exactly once**, in true or complemented form.
 - Example
 - $J(a,b,c) = (a + b + c')$ is a maxterm
 - $J(a,b,c) = (a+b)$ is not a maxterm
 - $H(a,b,c) = (a + b + c')(a' + b' + c')$ is a product of maxterms

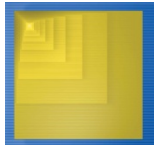


Minterms and Maxterms for a 3-variable function

$F(X,Y,Z)$

- A **minterm** is a product term that is 1 in exactly one row of the truth table.
- A **maxterm** is a sum term that is 0 in exactly one row of the truth table.
- Important equation:
- $(\text{Minterm on } i\text{'th row})' = (\text{Maxterm on } i\text{th row})$

<i>Row</i>	<i>X</i>	<i>Y</i>	<i>Z</i>	<i>F</i>	<i>Minterm</i>	<i>Maxterm</i>
0	0	0	0	$F(0,0,0)$	$X' \cdot Y' \cdot Z'$	$X + Y + Z$
1	0	0	1	$F(0,0,1)$	$X' \cdot Y' \cdot Z$	$X + Y + Z'$
2	0	1	0	$F(0,1,0)$	$X' \cdot Y \cdot Z'$	$X + Y' + Z$
3	0	1	1	$F(0,1,1)$	$X' \cdot Y \cdot Z$	$X + Y' + Z'$
4	1	0	0	$F(1,0,0)$	$X \cdot Y' \cdot Z'$	$X' + Y + Z$
5	1	0	1	$F(1,0,1)$	$X \cdot Y' \cdot Z$	$X' + Y + Z'$
6	1	1	0	$F(1,1,0)$	$X \cdot Y \cdot Z'$	$X' + Y' + Z$
7	1	1	1	$F(1,1,1)$	$X \cdot Y \cdot Z$	$X' + Y' + Z'$



Example

row	x	y	F	F'	minterm	maxterm
0	0	0	1	0	$x'y'$	$x+y$
1	0	1	0	1	$x'y$	$x+y'$
2	1	0	0	1	xy'	$x'+y$
3	1	1	0	1	xy	$x'+y'$

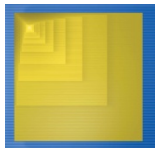
Are the following two the same?

$$\begin{aligned}
 \text{SOM } F &= x'y' & \text{POM } F &= (x+y')(x'+y)(x'+y') \\
 & & &= (xx' + xy + x'y' + yy')(x' + y') \\
 & & &= (xy + x'y')(x' + y') \\
 & & &= (xyx' + xyy' + x'y'x' + x'y'y') \\
 & & &= x'y'
 \end{aligned}$$

Alternatively,

Write the complement of the function F using minterms then take its complement

$$F' = x'y + xy' + xy \rightarrow F = (x+y')(x'+y)(x'+y')$$



Compact Sum of Minterm Representation

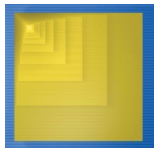
Compact Sum of Minterms (also known as “on-set”)

- List each minterm as a number, with each minterm's number determined from the binary representation of its variables' values.

- **Example:** Given sum of minterms for H determine its on-set.

- $H(a,b,c,d,e) = a'bcde + abcde + abcde,$
- $a'bcde$ corresponds to 01111 or 15,
- $abcde'$ corresponds to 11110 or 30 ,
- $abcde$ corresponds to 11111 or 31
- $H = \sum(15, 30, 31)$

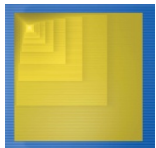
– $F = \sum_{X,Y,Z} (0,3,4,6,7) = X' \cdot Y' \cdot Z' + X' \cdot Y \cdot Z + X \cdot Y' \cdot Z' + X \cdot Y \cdot Z' + X \cdot Y \cdot Z$



Compact Product of Maxterm Representation

Compact Product of Maxterms (aka. **off-set**)

- **Example:** Given product of maxterms $F = (X+Y+Z') \cdot (X+Y'+Z) \cdot (X'+Y+Z')$
- Use the on-set to find the off-set
- complement of $(X'+Y+Z')$ is $X Y' Z$ and corresponds to 101 or 5, similarly for the others
- $F = \prod_{X,Y,Z} (1,2,5) = (X+Y+Z') \cdot (X+Y'+Z) \cdot (X'+Y+Z')$



Conversion from minterm/maxterm lists

- It is easy to convert between a minterm list and a maxterm list.
- For a function of n variables, the minterms and maxterms are in the set $\{0, 1, \dots, 2^n-1\}$.
- A minterm or maxterm list contains a subset of these numbers.
- **To switch between the lists, one takes the set complement.**

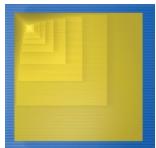
- **Examples:**

I. $F = \sum_{x,y} (0) = \prod_{x,y} (1,2,3)$
 $x'y' = (x+y')(x'+y)(x'+y')$

row	x	y	F	F'	minterm	maxterm
0	0	0	1	0	$x'y'$	$x+y$
1	0	1	0	1	$x'y$	$x+y'$
2	1	0	0	1	xy'	$x'+y$
3	1	1	0	1	xy	$x'+y'$

Write the complement of the function F using minterms then take its complement

$$F' = x'y + xy' + xy \rightarrow F = (x+y')(x'+y)(x'+y')$$



Examples

I. $\sum_{A,B,C} (0,1,2,3) = \prod_{A,B,C} (4,5,6,7)$

II. $\sum_{W,X,Y,Z} (1,2,3,5,8,12,13) = \prod_{W,X,Y,Z} (0,4,6,7,9,10,11,14,15)$



Exercise

- Generate a sum of minterms expression for a comparator.

INPUTS: $x, y \in \{0, 1, 2, 3\}$
 OUTPUT: $z \in \{G, E, S\}$

Encoding for x and y:
 $x = 2x_1 + x_0$ $y = 2y_1 + y_0$

FUNCTION: $z = \begin{cases} G & \text{if } x > y \\ E & \text{if } x = y \\ S & \text{if } x < y \end{cases}$

G: greater
 E: equal
 S: smaller

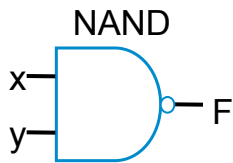
		y			
		0	1	2	3
x	0	E	S	S	S
	1	G	E	S	S
	2	G	G	E	S
	3	G	G	G	E

z

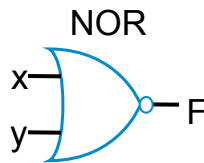
Encoding for z:

z	$z_2 z_1 z_0$
G	100
E	010
S	001

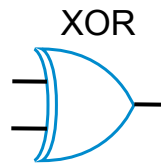
More Gates



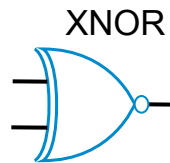
x	y	F
0	0	1
0	1	1
1	0	1
1	1	0



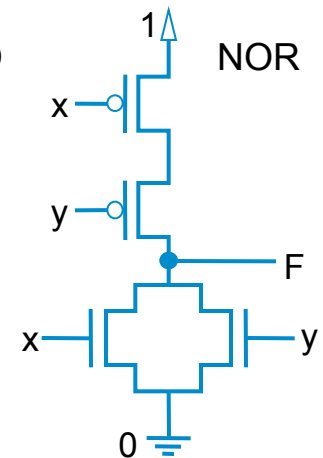
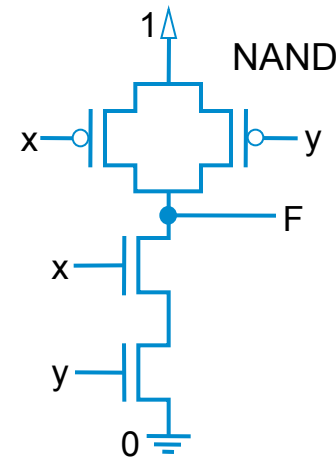
x	y	F
0	0	1
0	1	0
1	0	0
1	1	0



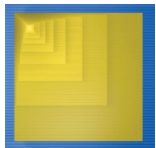
x	y	F
0	0	0
0	1	1
1	0	1
1	1	0



x	y	F
0	0	1
0	1	0
1	0	0
1	1	1

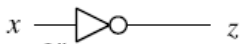
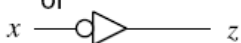
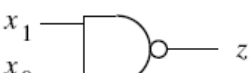


- NAND: Opposite of AND (“NOT AND”)
- NOR: Opposite of OR (“NOT OR”)
- XOR: Exactly 1 input is 1, for 2-input XOR. (For more inputs -- odd number of 1s)
- XNOR: Opposite of XOR (“NOT XOR”)
- NAND same as AND with power & ground switched
 - Why? nMOS conducts 0s well, but not 1s (reasons beyond our scope) -- so NAND more efficient
- Likewise, NOR same as OR with power/ground switched
- AND in CMOS: NAND with NOT
- OR in CMOS: NOR with NOT
- So NAND/NOR more common



Gates

3 input XOR

Gate type	Symbol	Switching expression
NOT		$z = x'$
		
AND		$z = x_1 x_0$
OR		$z = x_1 + x_0$
NAND		$z = (x_1 x_0)'$
NOR		$z = (x_1 + x_0)'$
XOR		$z = x_1 x_0' + x_1' x_0$ $= x_1 \oplus x_0$
XNOR		$z = x_1' x_0' + x_1 x_0$

A	B	C	A xor B xor C
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

More Gates: Example Uses

- Detecting all 0s (count down from 7 to 0, sound alarm when 0)
 - Use NOR
- Detecting equality
 - Use XNOR
- Detecting odd number of 1's
 - Use XOR
 - Useful for **generating** even “parity” bit (parity bit is 1 if there are odd number of information bits, that is, code not even)
 - Detecting error for even parity (XORing incoming bits, 1 indicates error, that is, code not even)

